Q1(a)

XML	JSON
Uses tags like <name>John</name>	Uses key-value pairs like "name": "John"
More verbose and larger in size	Lightweight and compact
Supports attributes and namespaces	Does not support attributes and namespaces
Difficult to read and write compared to JSON	Easy to read and write
Mainly used for document storage and data exchange	Mainly used for APIs and web applications
Parsing is slower	Parsing is faster
Supports comments	Does not officially support comments
Data is stored in tree structure using tags	Data is stored as objects and arrays
Requires closing tags	Does not require closing tags
More secure with schema validation support	Less strict compared to XML validation
Uses XML parsers	Uses JSON parsers
Supports mixed content (text + tags)	Does not support mixed content easily
Older technology	Newer and more popular in modern web development
File size is generally larger	File size is generally smaller
Schema definition is done using XSD or DTD	Schema definition is usually done using JSON Schema

B)

Line Number	Answer
3	Element
5	SubElement
9	ElementTree
10	Write
13	Parse
14	Getroot
17	Find

Q 2)

```
small_count = 0
medium_count = 0
tall_count = 0
valid_trees = 0

for i in range(6):

    tree_name = input("Enter Tree Name: ")

    if tree_name.lower() == "stop":
        print("Data entry stopped by user.")
        Break

    height = float(input("Enter Height of Tree (in meters): "))
```

```
    age = int(input("Enter Age of Tree: "))

    # Skip invalid height
    if height <= 2:
        print("Invalid tree height. Skipping record.")
        Continue

    valid_trees += 1

    # Classification based on height
    if height < 5:
        print(tree_name, "-> Small Tree")
        small_count += 1

    elif height >= 5 and height <= 10:
        print(tree_name, "-> Medium Tree")
        medium_count += 1

    else:
        print(tree_name, "-> Tall Tree")
        tall_count += 1

# Final Counts
print("\nTree Classification Summary")
print("Small Trees:", small_count)
print("Medium Trees:", medium_count)
print("Tall Trees:", tall_count)
print("Total Valid Trees:", valid_trees)
```

Q 3)

Q. No.	Task	Possible Single-Line Solutions
1	Creating a list	my_list = [1,2,3] my_list = list((1,2,3)) my_list = list(range(5)) my_list = ["a","b","c"]
2	Finding length of a list	len(my_list) print(len(my_list)) size = len(my_list)
3	Adding elements to a set	my_set.add(10) my_set.update([10,20]) `my_set
4	Displaying elements of a tuple	print(my_tuple) for i in my_tuple: print(i) print(*my_tuple) tuple(map(print, my_tuple))

5	Creating a dictionary	my_dict = {"a":1, "b":2} my_dict = dict(a=1, b=2) my_dict = dict([("a",1),("b",2)]) my_dict = {1:"One", 2:"Two"}
6	Finding maximum element from a list	max(my_list) print(max(my_list)) largest = max(my_list) sorted(my_list)[-1]

Q4(a) Any three

SOAP	REST
Protocol	Architectural Style
Uses XML only	Uses JSON, XML etc.
Heavyweight	Lightweight
Slower	Faster
More secure	Less secure
Stateful	Stateless
Complex	Simple
Uses more bandwidth	Uses less bandwidth
Better for enterprise applications	Better for web APIs

Q4(b)

10. dumps
11. write
12. load

Q5

```
# Q5

import pandas as pd
import matplotlib.pyplot as plt

data = {
   "RollNo":[101,102,103,104,105],
   "Name":["Aman","Riya","Karan","Simran","Arjun"],
   "Python":[85,92,None,88,None],
```

```
    "AI":[78,None,75,91,65],
    "DataScience":[90,95,None,85,70]
}

# 1. Create DataFrame
df = pd.DataFrame(data)

print("DataFrame:")
print(df)

# 2. Find Null Values
print("\nNull Values:")
print(df.isnull())

# 3. Treat Missing Values
df.fillna(df.mean(numeric_only=True), inplace=True)

print("\nAfter Treating Missing Values:")
print(df)

# 4. Calculate Percentage
df["Percentage"] = (
    df["Python"] +
    df["AI"] +
    df["DataScience"]
) / 3

print("\nStudents Scoring Above 80%:")
print(df[df["Percentage"] > 80])

# 5. Sort According to Percentage
df = df.sort_values(by="Percentage")

print("\nSorted Data:")
print(df)

# 6. Plot Bar Graph
df.plot(
    x="Name",
    y=["Python","AI","DataScience"],
    kind="bar"
)

plt.title("Subject-wise Marks")
plt.xlabel("Students")
plt.ylabel("Marks")

plt.show()
```

Q6

```
# Q6

num = int(input("Enter Number: "))

temp = num
reverse = 0

while num > 0:

    digit = num % 10

    reverse = reverse * 10 + digit

    num = num // 10

if temp == reverse:
    print("Palindrome Number")

else:
    print("Not Palindrome Number")
```

Q7(a)

```
# Q7(a)

class InvalidAmount(Exception):
    pass

try:

    balance = 10000

    amount = int(input("Enter Withdrawal Amount: "))

    if amount <= 0:
        raise InvalidAmount("Invalid Withdrawal Amount")

    divisor = int(input("Enter Divisor: "))

    result = balance / divisor

    print("Division Result =", result)

    print("Remaining Balance =", balance - amount)

except ValueError:
    print("Invalid Input")
```

```
except ZeroDivisionError:
    print("Cannot Divide By Zero")

except InvalidAmount as e:
    print(e)

finally:
    print("Transaction Completed")
```

Q7(b)

```
# Q7(b)

try:

    file = open("sales.txt", "w")

    total_sales = 0
    valid_records = 0

    for i in range(3):

        product = input("Enter Product Name: ")

        sales = int(input("Enter Sales Amount: "))

        if sales <= 0:
            print("Invalid Record")
            continue

        file.write(product + " " + str(sales) + "\n")

        total_sales = total_sales + sales

        valid_records = valid_records + 1

    print("Current Position:", file.tell())

    file.close()

    file = open("sales.txt", "r")

    file.seek(0)

    print("\nFile Data:")
    print(file.read())

    print("Total Sales =", total_sales)
```

```
    print("Valid Records =", valid_records)

    file.close()

except FileNotFoundError:
    print("File Not Found")

except Exception as e:
    print(e)
```